Theme bundles apply to HTML and paged output. Select a theme with `--theme`, `theme` in `calepin.toml`, or `#calepin.setup(theme: ...)`:

```
calepin compile paper.typ --theme calepin
```

`--theme` is optional. Builds use the builtin `calepin` bundle by default. The builtin `academic` bundle customizes website pages and falls back to `calepin` for single-document HTML and paged output.

Local themes are selected by pointing `--theme` or `theme` at a theme bundle:

```
themes/my-theme/
  paged.typ
  document.html
  site.html
  partials/
  styles/
  scripts/
```

`document.html` and `site.html` are MiniJinja templates. `partials/` files can be included with `{% include "partials/name.html" %}`. CSS files in `styles/` and JavaScript files in `scripts/` are loaded in filename order and exposed as `styles` and `scripts` arrays.

Templates can access:

- `doc.head`
- `doc.body_open`
- `doc.body`
- `doc.body_close`
- `doc.title`
- `site.sidebar`
- `site.sidebar_sections`
- `site.navbar_left`
- `site.navbar_center`
- `site.navbar_right`
- `site.languages`
- `site.translations`
- `site.language`
- `site.toc`
- `site.title`
- `site.description`
- `site.base_url`
- `site.logo`
- `site.logo_alt`
- `site.home_url`
- `site.github_url`
- `site.current_url`
- `site.page_title`
- `snippets.css.theme`
- `snippets.css.code`
- `snippets.css.widgets`
- `snippets.js.copy_code`
- `snippets.js.language_picker`
- `snippets.js.theme_toggle`

- `snippets.typst.code_block`
- `styles`
- `scripts`
- `syntax_css`
- `theme`
- `target`

Navigation entries expose `href`, `label`, `label_html`, `active`, and `widget`. Ordinary links have `widget = none`; widget entries preserve the configured string. The bundled themes understand `widget = "theme"` and `widget = "language"`, and custom themes can define their own widget names.

Bundled snippets are small reusable pieces that can be used across local themes. For example, an HTML theme can add shared base styling, widget styling, code/output styling, and widget behavior without maintaining its own CSS or JavaScript:

```
<style>{{ snippets.css.theme }}</style>
<style>{{ snippets.css.code }}</style>
<style>{{ snippets.css.widgets }}</style>
<script>{{ snippets.js.copy_code }}</script>
<script>{{ snippets.js.language_picker }}</script>
<script>{{ snippets.js.theme_toggle }}</script>
```

`snippets.css.theme` is the shared visual base used by `calepin` and `academic`. It defines common typography, heading sizes, accent variables, Pico primary colors, code/output variables, figure defaults, and global document defaults. Theme-specific CSS should generally be limited to the HTML shell and layout differences that cannot be shared.

`snippets.css.widgets` pairs with the shared JavaScript widgets:

- `snippets.js.theme_toggle` enhances controls marked with `data-calepin-theme-toggle`
- `snippets.js.language_picker` enhances selects marked with `data-calepin-language-picker`

Use these snippets in custom HTML themes to keep the dark-mode control, language picker, code blocks, and base typography consistent with the bundled themes.

`paged.typ` is a Typst file, not a MiniJinja template. The bundled `calepin` theme's `paged.typ` is enabled by default for `calepin compile` and website PDF/SVG/PNG builds. Customize it by ejecting a bundle:

```
calepin new theme
```

The default `paged.typ` imports `/.calepin/snippets/typst/code-block.typ` and applies it to raw source blocks. Custom paged themes can import the same general snippet:

```
#import "/.calepin/snippets/typst/code-block.typ": code-block
```

Set `theme = false` or use an empty `paged.typ` to disable paged styling.